

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Костромской государственный университет»

Институт: Высшая ИТ-школа

Кафедра: Информационные системы и технологии

Направление: Информационные технологии в бизнесе

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

на тему:

**РАЗРАБОТКА КРОССПЛАТФОРМЕННОГО ВЕБ-ПРИЛОЖЕНИЯ
«ЛИЧНЫЙ СЕКРЕТАРЬ» С ИСПОЛЬЗОВАНИЕМ ИИ**

Исполнитель:

студент группы 22-ИСбо-4

Рачинский Иван Александрович

Руководитель:

Мозохин Александр Евгеньевич

Кострома, 2026

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ

Тема выпускной квалификационной работы: «Разработка кроссплатформенного веб-приложения «Личный секретарь» с использованием ИИ». Исполнитель: Рачинский Иван Александрович, группа 22-ИСбо-4. Руководитель: Мозохин Александр Евгеньевич. Срок выполнения работы соответствует календарному графику подготовки ВКР.

Цель задания состоит в разработке и описании кроссплатформенного веб-приложения «Личный секретарь», получившего рабочее название MindVault. Работа должна включать анализ предметной области, обзор аналогов, формирование требований, проектирование архитектуры и модели данных, реализацию клиентской и серверной частей, тестирование и подготовку демонстрационной версии.

В рамках задания необходимо выполнить анализ разрозненного хранения пользовательских данных, определить границы MVP, выбрать инструменты разработки, спроектировать пользовательский интерфейс и программную архитектуру, реализовать работу с файлами, заметками, списками, напоминаниями, папками и ИИ-ассистентом.

Особое внимание требуется уделить логике различения обычного вопроса ассистенту и команды на сохранение объекта. Подтверждение операции должно появляться только после успешной записи данных на сервере. Спорные вводы должны обрабатываться без автоматического создания объектов, если намерение пользователя недостаточно явно.

Таблица 1 - Календарный план выполнения ВКР

Этап	Содержание работы	Результат
1	Анализ предметной области и аналогов	Сформулированы проблема, ограничения существующих решений и место приложения «Личный секретарь»
2	Формирование требований и границ MVP	Подготовлены user story, функциональные и нефункциональные требования
3	Выбор технологий	Проверен фактический стек проекта: React, Vite, TypeScript, Express, PostgreSQL, Drizzle ORM, Docker Compose
4	Проектирование системы	Подготовлены архитектурные схемы, модель данных, алгоритм обработки сообщений и структура интерфейса
5	Реализация подсистем	Описаны клиентская часть, API, база данных, файловое хранилище, папки, заметки, списки, напоминания и ассистент
6	Тестирование и подготовка демонстрации	Проверены основные сценарии, зафиксированы ограничения MVP, подготовлены скриншоты интерфейса

АННОТАЦИЯ

Выпускная квалификационная работа выполнена на тему «Разработка кроссплатформенного веб-приложения «Личный секретарь» с использованием ИИ». Объектом исследования является организация личных цифровых материалов пользователя: текстовых сообщений, файлов, заметок, списков и напоминаний. Предметом исследования являются методы проектирования и реализации кроссплатформенного веб-приложения, объединяющего персональное хранилище и ИИ-ассистента.

Цель работы заключается в разработке веб-приложения «Личный секретарь», которое предоставляет пользователю единый чатовый вход для сохранения и обработки информации. В рамках ВКР разработано кроссплатформенное веб-приложение «Личный секретарь», получившее рабочее название MindVault.

В работе использованы методы анализа предметной области, сравнительного анализа аналогов, проектирования информационных систем, проектирования пользовательского интерфейса, программной реализации, функционального тестирования и анализа ограничений MVP. Практическим результатом является демонстрационная версия приложения с клиентской частью, серверным API, базой данных, загрузкой файлов, папками, заметками, списками, напоминаниями и ассистентом.

Практическая значимость работы состоит в том, что разработанное приложение показывает возможность объединения разрозненных пользовательских материалов в одном рабочем пространстве и создает основу для дальнейшего развития персонального ассистента с семантическим поиском, улучшенным чтением документов и календарными интеграциями.

Ключевые слова: КРОССПЛАТФОРМЕННОЕ ВЕБ-ПРИЛОЖЕНИЕ, ЛИЧНЫЙ СЕКРЕТАРЬ, MINDVAULT, ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ, ФАЙЛОВОЕ ХРАНИЛИЩЕ, ЗАМЕТКИ, СПИСКИ, НАПОМИНАНИЯ, ЧАТОВЫЙ ИНТЕРФЕЙС, API, БАЗА ДАННЫХ, UX/UI.

РЕФЕРАТ

Выпускная квалификационная работа содержит 56 страниц, 21 таблицу, 15 рисунков и 27 использованных источников.

Ключевые слова: кроссплатформенное веб-приложение, личный секретарь, MindVault, искусственный интеллект, файловое хранилище, заметки, списки, напоминания, чатовый интерфейс, API, база данных, UX/UI.

Цель работы - разработать кроссплатформенное веб-приложение «Личный секретарь» с использованием ИИ. Для достижения цели выполнены анализ предметной области, обзор аналогов, формирование требований, выбор инструментов, проектирование архитектуры и модели данных, реализация основных подсистем, тестирование и подготовка демонстрационной версии.

Методы выполнения работы включают анализ пользовательских сценариев, сравнение технологических вариантов, проектирование клиент-серверной архитектуры, моделирование данных, разработку REST API, организацию контейнерного запуска, функциональное тестирование и проверку адаптивного интерфейса.

Результатом работы является приложение MindVault, реализующее чат, хранение файлов, заметки, списки, напоминания, папки и взаимодействие с ассистентом. Область применения - личная организация информации, учебные и рабочие материалы, подготовка документов, хранение быстрых мыслей и задач.

ОГЛАВЛЕНИЕ

Задание на выпускную квалификационную работу

Аннотация

Реферат

Перечень сокращений

Введение

Глава 1. Анализ задачи и обзор аналогов

- 1.1. Описание предметной области
- 1.2. Проблема разрозненного хранения пользовательских данных
- 1.3. Обзор существующих решений
- 1.4. Сравнительный анализ аналогов
- 1.5. Выводы по главе

Глава 2. Требования к системе и выбор инструментов

- 2.1. Цель и границы MVP
- 2.2. User story
- 2.3. Функциональные требования
- 2.4. Нефункциональные требования
- 2.5. Приоритизация требований
- 2.6. Критерии выбора инструментов
- 2.7. Сравнение технологических вариантов
- 2.8. Выводы по главе

Глава 3. Проектирование веб-приложения «Личный секретарь» / MindVault

- 3.1. Общая архитектура системы
- 3.2. Диаграмма развертывания
- 3.3. Event modelling / timeline событий
- 3.4. Архитектура frontend
- 3.5. Архитектура backend
- 3.6. Модель данных
- 3.7. Проектирование обработки сообщений
- 3.8. Проектирование работы с файлами
- 3.9. Проектирование интерфейса

3.10. Выводы по главе

Глава 4. Реализация системы

- 4.1. Структура проекта
- 4.2. Реализация клиентской части
- 4.3. Реализация серверной части
- 4.4. Реализация базы данных
- 4.5. Реализация файлового хранилища
- 4.6. Реализация папок
- 4.7. Реализация заметок, списков и напоминаний
- 4.8. Реализация логики ассистента
- 4.9. Реализация адаптивного интерфейса
- 4.10. Развертывание и запуск
- 4.11. Сложности при разработке и принятые решения
- 4.12. Выводы по главе

Глава 5. Тестирование и анализ результатов

- 5.1. Методика тестирования
- 5.2. Функциональное тестирование
- 5.3. Тестирование интерфейса
- 5.4. Тестирование адаптивности
- 5.5. Тестирование ассистента
- 5.6. Тестирование нестандартных вводов
- 5.7. Ограничения текущей версии
- 5.8. Сравнение результата с требованиями
- 5.9. Выводы по главе

Заключение

Список использованных источников

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ

ВКР	выпускная квалификационная работа
ИС	информационная система
ИИ	искусственный интеллект
API	программный интерфейс приложения
UI	пользовательский интерфейс
UX	пользовательский опыт
БД	база данных
CRUD	базовые операции создания, чтения, изменения и удаления данных
MVP	минимально жизнеспособный продукт
SPA	одностраничное веб-приложение
HTTP	протокол передачи гипертекста
HTTPS	защищенный протокол передачи гипертекста
ORM	объектно-реляционное отображение
LLM	большая языковая модель
JSON	формат обмена данными
SQL	язык структурированных запросов
Docker	платформа контейнеризации

ВВЕДЕНИЕ

Личные цифровые материалы пользователя сегодня редко хранятся в одном месте. Файлы оказываются в облачном диске, быстрые мысли - в избранных сообщениях мессенджера, списки дел - в отдельном планировщике, длинные заметки - в редакторе документов, а вопросы к искусственному интеллекту - в отдельном чате. Каждый из этих инструментов полезен сам по себе, но в повседневной работе пользователь вынужден постоянно переключаться между ними.

Проблема проявляется особенно заметно в учебной и проектной деятельности. Пользователь сохраняет файл с заданием, делает заметку по встрече, отправляет себе ссылку, просит ассистента объяснить документ и ставит напоминание проверить результат. Все эти действия относятся к одной задаче, но технически находятся в разных приложениях. Из-за этого теряется контекст, сложнее найти нужный материал и выше вероятность забыть важное действие.

Telegram Saved Messages удобен как быстрый буфер, однако он не является полноценной информационной системой: там нет строгой модели заметок, списков, напоминаний и файловых карточек. Google Drive и Яндекс Диск хорошо решают задачу хранения файлов, но не дают естественного чатового входа для быстрых мыслей и не связывают файлы с задачами. Notion и Obsidian сильны как базы знаний, однако требуют дисциплины структурирования и не всегда удобны для мгновенного сохранения с телефона.

Отдельный ИИ-ассистент, например ChatGPT, помогает анализировать текст, но обычно не знает, какие файлы, заметки и напоминания пользователь сохранил в своем рабочем пространстве. Поэтому он хорошо отвечает на вопрос, но хуже работает как личный секретарь, который должен понимать контекст пользователя, опираться на сохраненные материалы и не путать обычный вопрос с командой создать объект.

Актуальность темы «Разработка кроссплатформенного веб-приложения «Личный секретарь» с использованием ИИ» связана с потребностью в едином интерфейсе, который объединяет быстрый ввод, хранение и

интеллектуальную обработку личной информации. Такой интерфейс должен быть доступен с настольного компьютера и мобильного устройства, так как пользователь сохраняет данные не только за рабочим столом, но и в дороге, на занятиях, на встрече или во время подготовки к защите.

В данной работе под «личным секретарем» понимается не отдельный чат-бот, а веб-приложение, где ассистент встроен в структуру пользовательских данных. Он должен отвечать на вопросы, помогать с файлами и текстом, но при этом не должен создавать заметки или напоминания без явного намерения пользователя. Это требование важно с инженерной точки зрения: если система подтверждает действие до записи в базу данных, пользователь получает ложное ощущение надежности.

В рамках работы разработано кроссплатформенное веб-приложение «Личный секретарь», получившее рабочее название MindVault. Рабочее название используется далее для удобства описания интерфейса, структуры проекта и демонстрационной версии, однако официальной темой ВКР остается разработка приложения «Личный секретарь» с использованием ИИ.

Объектом исследования является процесс организации личной цифровой информации пользователя. Предметом исследования является проектирование и реализация кроссплатформенного веб-приложения, которое объединяет чатовый интерфейс, файловое хранилище, заметки, списки, напоминания, папки и ИИ-ассистента.

Цель работы - разработать кроссплатформенное веб-приложение «Личный секретарь» с использованием ИИ, проверить его на основных пользовательских сценариях и подготовить демонстрационную версию. Для достижения цели необходимо проанализировать предметную область, рассмотреть аналоги, сформулировать требования, выбрать инструменты, спроектировать архитектуру, реализовать основные подсистемы и провести тестирование.

Методическую основу работы составляют анализ требований, сравнительный анализ аналогов, проектирование информационных систем, разработка клиент-серверного веб-приложения, проектирование пользовательского интерфейса, функциональное тестирование и проверка

адаптивности. При оформлении документа учитывались правила КГУ и требования к структуре отчетных документов [1], [3].

Практическая значимость работы заключается в создании прототипа, который демонстрирует единый сценарий работы с личными материалами. Приложение может быть использовано как учебный проект, как основа для дальнейшей разработки персональной базы знаний и как демонстрационный пример интеграции пользовательских данных с ассистентом.

ВКР состоит из введения, пяти глав, заключения и списка источников. В первой главе анализируется предметная область и существующие решения. Во второй главе формулируются требования и обосновывается технологический выбор. В третьей главе описывается проектирование приложения. В четвертой главе рассматривается реализация. В пятой главе приводятся тестирование, ограничения и анализ результата.

ГЛАВА 1. АНАЛИЗ ЗАДАЧИ И ОБЗОР АНАЛОГОВ

1.1. Описание предметной области

Предметная область приложения «Личный секретарь» находится на пересечении персональных информационных систем, файловых хранилищ, заметочников, планировщиков и ассистентов на основе языковых моделей. Пользовательская задача не сводится только к хранению файлов или только к ведению задач: в реальной работе один и тот же материал может быть файлом, заметкой, списком действий и поводом задать вопрос ассистенту.

Для учебного проекта особенно важна скорость фиксации информации. Пользователь может получить тему, ссылку, документ или идею в момент, когда нет времени оформлять полноценную запись. Если система требует сразу выбрать раздел, заполнить много полей и решить, где хранить объект, пользователь возвращается к самому простому варианту: отправляет сообщение самому себе. Поэтому одним из ключевых требований становится единый вход через чат.

При этом единый вход не означает хаотичное сохранение всего подряд. Чат должен различать обычный вопрос, команду создать заметку, команду создать список и команду поставить напоминание. Ошибка в этой логике приводит к двум проблемам: либо пользовательские материалы засоряются случайными вопросами, либо важная команда не выполняется. Поэтому обработка намерений является центральной частью предметной области.

Вторая особенность предметной области - работа с контекстом. Ассистент полезен не только тогда, когда отвечает на общий вопрос, но и тогда, когда опирается на сохраненные данные пользователя: файл с заданием, заметку с идеей, список подготовки, будущие напоминания. Такой подход ближе к персональному рабочему кабинету, чем к обычному чат-боту.

Кроссплатформенность в рамках проекта достигается за счет веб-интерфейса. Приложение не требует отдельной установки для каждой операционной системы: пользователь открывает его в браузере на компьютере или смартфоне. Это решение соответствует границам ВКР и позволяет сосредоточиться на функциональности, а не на разработке отдельных нативных клиентов.

1.2. Проблема разрозненного хранения пользовательских данных

Разрозненное хранение возникает не потому, что существующие сервисы плохи, а потому, что каждый из них оптимизирован под свой тип данных. Облачный диск удобен для документов, но плохо подходит для быстрых мыслей. Планировщик задач удобен для сроков, но не хранит контекст файла. Заметочник удобен для текста, но требует ручной структуры. Чат с ИИ удобен для вопросов, но не является хранилищем пользовательских объектов.

На практике пользователь часто выбирает кратчайший путь. Если нужно быстро сохранить мысль, он отправляет ее в мессенджер. Если нужно передать документ ассистенту, он открывает отдельный сервис ИИ. Если нужно не забыть задачу, он создает напоминание в календаре или планировщике. Через несколько дней становится трудно восстановить связь между этими действиями.

Для проекта MindVault проблема формулируется как необходимость единого интерфейса, где разные типы данных сохраняются в общей системе и могут быть найдены или использованы ассистентом. Это не означает попытку заменить полноценные профессиональные инструменты. MVP должен проверить более узкую гипотезу: пользователю полезен единый рабочий вход, который принимает текст, файлы и команды, а затем раскладывает их по понятным разделам.

Важным ограничением является доверие к действиям ассистента. Если ассистент пишет «готово», но объект фактически не создан в базе данных, пользователь может пропустить важную задачу. Поэтому подтверждение действия должно быть связано с результатом backend-операции, а не только с текстовым ответом модели.

Еще одна часть проблемы связана с мобильным использованием. Если приложение хорошо работает только на широком экране, оно не решает задачу быстрого сохранения. Пользователь должен иметь возможность открыть чат, отправить сообщение, посмотреть файл или напоминание с телефона. Поэтому интерфейс проектировался как адаптивный.

1.3. Обзор существующих решений

Google Drive представляет собой облачное файловое хранилище. Для задачи «Личный секретарь» полезны следующие возможности: хранение документов, папки, совместный доступ, предпросмотр файлов. Основное преимущество решения - надежное файловое хранилище и интеграция с офисными документами. Ограничение для рассматриваемой ВКР состоит в том, что не является единым чатовым входом и не решает задачу заметок, списков и напоминаний в одном сценарии.

Яндекс Диск представляет собой облачное файловое хранилище. Для задачи «Личный секретарь» полезны следующие возможности: хранение файлов, синхронизация, папки, просмотр медиа. Основное преимущество решения - понятная модель хранения и доступность с разных устройств. Ограничение для рассматриваемой ВКР состоит в том, что не связывает файлы с личными задачами и ассистентом внутри единого интерфейса.

Dropbox представляет собой облачное хранилище и сервис синхронизации. Для задачи «Личный секретарь» полезны следующие возможности: файлы, папки, общий доступ, история версий. Основное преимущество решения - надежная синхронизация и развитая экосистема. Ограничение для рассматриваемой ВКР состоит в том, что ориентирован на файлы, а не на чат, заметки и команды сохранения.

Notion представляет собой универсальная база знаний и рабочее пространство. Для задачи «Личный секретарь» полезны следующие возможности: страницы, базы данных, шаблоны, совместная работа, AI-функции. Основное преимущество решения - гибкая структура и широкие возможности организации знаний. Ограничение для рассматриваемой ВКР состоит в том, что требует ручного моделирования пространства и сложнее для мгновенного сохранения простых сообщений.

Obsidian представляет собой локальная база знаний на Markdown. Для задачи «Личный секретарь» полезны следующие возможности: связанные заметки, граф, плагины, локальные файлы. Основное преимущество решения - контроль над данными и удобная работа с текстовыми заметками. Ограничение для рассматриваемой ВКР состоит в том, что слабее подходит для файлового кабинета, напоминаний и чатового сценария с ассистентом.

Evernote представляет собой заметочник и персональный архив. Для задачи «Личный секретарь» полезны следующие возможности: заметки, блокноты, вложения, поиск. Основное преимущество решения - хорош для текстовых записей и быстрых заметок. Ограничение для рассматриваемой ВКР состоит в том, что не закрывает сценарий единого ИИ-чата и работы с разными типами объектов.

Telegram Saved Messages представляет собой личный чат в мессенджере. Для задачи «Личный секретарь» полезны следующие возможности: быстрые сообщения, файлы, ссылки, поиск по чату. Основное преимущество решения - очень низкий порог входа и привычный чатовый формат. Ограничение для рассматриваемой ВКР состоит в том, что нет строгих сущностей заметок, списков и напоминаний, сложно поддерживать порядок.

Todoist представляет собой менеджер задач. Для задачи «Личный секретарь» полезны следующие возможности: задачи, проекты, сроки, фильтры, уведомления. Основное преимущество решения - сильная работа с задачами и сроками. Ограничение для рассматриваемой ВКР состоит в том, что не является файловым хранилищем и не объединяет разные материалы в одном чате.

ChatGPT представляет собой отдельный ИИ-ассистент. Для задачи «Личный секретарь» полезны следующие возможности: ответы на вопросы, генерация текста, анализ вложений в доступных тарифах. Основное преимущество решения - сильные возможности обработки естественного языка. Ограничение для рассматриваемой ВКР состоит в том, что не хранит пользовательские заметки, списки, файлы и напоминания как структуру конкретного приложения.

Таблица 2 - Характеристика аналогов

Решение	Назначение	Модель	Полезные функции	Ограничение для задачи
Google Drive	облачное файловое хранилище	условно бесплатная модель с платным расширением объема	хранение документов, папки, совместный доступ, предпросмотр файлов	не является единым чатовым входом и не решает задачу заметок, списков и напоминаний в одном сценарии
Яндекс Диск	облачное файловое хранилище	условно бесплатная модель	хранение файлов, синхронизация, папки, просмотр медиа	не связывает файлы с личными задачами и ассистентом внутри единого интерфейса
Dropbox	облачное хранилище и сервис синхронизации	условно бесплатная модель	файлы, папки, общий доступ, история версий	ориентирован на файлы, а не на чат, заметки и команды сохранения
Notion	универсальная база знаний и рабочее пространство	условно бесплатная модель	страницы, базы данных, шаблоны, совместная работа, AI-функции	требует ручного моделирования пространства и сложнее для мгновенного сохранения простых сообщений
Obsidian	локальная база знаний на Markdown	бесплатная личная версия, платные синхронизация и публикация	связанные заметки, граф, плагины, локальные файлы	слабее подходит для файлового кабинета, напоминаний и чатового сценария с ассистентом
Evernote	заметочник и персональный архив	условно бесплатная модель	заметки, блокноты, вложения, поиск	не закрывает сценарий единого ИИ-чата и работы с разными типами объектов
Telegram Saved Messages	личный чат в мессенджере	бесплатная модель	быстрые сообщения, файлы, ссылки, поиск по чату	нет строгих сущностей заметок, списков и напоминаний, сложно поддерживать порядок
Todoist	менеджер задач	условно бесплатная модель	задачи, проекты, сроки, фильтры, уведомления	не является файловым хранилищем и не объединяет разные материалы в одном чате
ChatGPT	отдельный ИИ-ассистент	условно бесплатная модель	ответы на вопросы, генерация текста, анализ вложений в доступных тарифах	не хранит пользовательские заметки, списки, файлы и напоминания как структуру конкретного приложения

1.4. Сравнительный анализ аналогов

Для сравнения были выбраны критерии, которые непосредственно связаны с задачей приложения «Личный секретарь». В таблице 2 отражены возможности хранения и структурирования данных, а в таблице 3 - наличие чатового входа, ИИ-обработки и пригодность решения для единого сценария.

Таблица 3 - Сравнение аналогов по хранению и структуре

Решение	Файлы	Заметки	Задачи	Папки
Google Drive	есть	через отдельные документы	нет	есть
Яндекс Диск	есть	ограниченно	нет	есть
Dropbox	есть	ограниченно	нет	есть
Notion	есть	есть	есть	есть
Obsidian	через локальные вложения	есть	через плагины	есть
Evernote	как вложения	есть	ограниченно	есть
Telegram Saved Messages	есть	как сообщения	нет	нет
Todoist	нет	ограниченно	есть	проекты
ChatGPT	зависит от режима	нет как БД	нет	нет

Таблица 4 - Сравнение аналогов по интеллектуальному сценарию

Решение	Чатовый вход	ИИ-обработка	Простота входа	Связь данных	Ограничение
Google Drive	нет	частично	средняя	файлы связаны папками	нет единого секретаря
Яндекс Диск	нет	частично	средняя	файлы связаны папками	нет заметок и команд
Dropbox	нет	частично	средняя	файлы связаны папками	нет личного чата
Notion	частично	есть	ниже для быстрого ввода	сильная	сложнее MVP-сценария
Obsidian	нет	через плагины	средняя	связи заметок	не фокус на напоминаниях
Evernote	нет	ограниченно	средняя	блокноты и теги	нет единого ассистента
Telegram Saved Messages	есть	нет	высокая	слабая	нет структурных сущностей
Todoist	частично	ограниченно	высокая	задачи и проекты	нет файлового кабинета
ChatGPT	есть	есть	высокая	нет общей БД приложения	нет CRUD-сущностей

Сравнение показывает, что существующие решения закрывают отдельные части задачи. Наиболее близкими по идее являются Notion, Telegram Saved Messages и ChatGPT, но каждое из них смещено в свою сторону: база знаний, быстрый чат или ассистент. Проект MindVault занимает промежуточное место: он не стремится заменить все возможности аналогов, а проверяет сценарий единого входа и контекстного ассистента.

1.5. Выводы по главе

В первой главе установлено, что задача ВКР связана не с созданием еще одного облачного диска или заметочника, а с объединением личных материалов в едином рабочем интерфейсе. Основная ценность приложения «Личный секретарь» состоит в связке «сохранить - найти - обработать - использовать».

Обзор аналогов показал, что отдельные решения хорошо выполняют свои функции, однако не дают цельного MVP-сценария, в котором

пользователь отправляет сообщение или файл в общий чат, а система сама понимает, является ли ввод вопросом, заметкой, списком или напоминанием.

Следовательно, в проекте необходимо реализовать не максимальную функциональность, а устойчивое ядро: чат, структурированные сущности, папки, файловое хранение, базовую работу ассистента и адаптивный интерфейс. Эта граница позволяет разработать демонстрационную версию без необоснованного усложнения архитектуры.

ГЛАВА 2. ТРЕБОВАНИЯ К СИСТЕМЕ И ВЫБОР ИНСТРУМЕНТОВ

2.1. Цель и границы MVP

Цель MVP заключается в проверке основной идеи приложения «Личный секретарь»: пользователь должен иметь один вход для быстрых сообщений, файлов, заметок, списков, напоминаний и вопросов ассистенту. В MVP не требуется реализовывать все функции полноценных облачных сервисов, командную работу или промышленную систему безопасности, но должны быть показаны основные сценарии.

Граница MVP была определена после анализа аналогов и фактической сложности проекта. В первую версию вошли регистрация и вход, основной чат, разделы файлов, заметок, списков, напоминаний, настройки, папки, загрузка файлов, хранение данных в PostgreSQL, взаимодействие с ассистентом и контейнерный запуск. Эти функции достаточны, чтобы продемонстрировать идею единого рабочего кабинета.

Вне границ MVP оставлены семантический поиск по векторным представлениям, OCR для изображений, совместная работа нескольких пользователей над одним пространством, календарные интеграции, роли доступа, мобильное нативное приложение и промышленная эксплуатация с доменом и HTTPS. Эти направления описаны в заключении как перспективы развития.

2.2. User story

Таблица 5 - User story приложения «Личный секретарь»

Сценарий	Формулировка
Сообщение в чат	как пользователь, я хочу отправить сообщение в чат, чтобы быстро сохранить мысль или задать вопрос
Загрузка файла	как пользователь, я хочу загрузить файл, чтобы хранить его вместе с другими материалами
Объяснение файла	как пользователь, я хочу попросить ассистента объяснить файл, чтобы быстрее понять содержание
Заметка из сообщения	как пользователь, я хочу создать заметку из сообщения, чтобы не оформлять ее вручную в отдельном редакторе
Список	как пользователь, я хочу создать список покупок или дел, чтобы вести простые чек-листы
Напоминание	как пользователь, я хочу создать напоминание, чтобы не забыть действие
Папки	как пользователь, я хочу разложить материалы по папкам, чтобы поддерживать порядок
Мобильный доступ	как пользователь, я хочу открыть приложение с телефона и не терять основные функции
Обычный вопрос	как пользователь, я хочу, чтобы вопрос ассистенту не сохранялся автоматически как заметка или напоминание

Эти истории показывают, что пользовательский путь строится вокруг быстрых действий. В отличие от традиционного файлового менеджера, пользователь не обязан начинать с выбора типа объекта. Он может начать с сообщения, а система должна либо ответить, либо предложить действие, либо выполнить явно указанную команду.

2.3. Функциональные требования

Таблица 6 - Функциональные требования

Код	Подсистема	Описание	Приоритет
F1	Чат	отправка текстовых сообщений, отображение истории, поддержка ответа ассистента	обязательное
F2	Файлы	загрузка файла, хранение имени, MIME-типа, размера, содержимого и текстового предпросмотра	обязательное
F3	Заметки	создание, просмотр, изменение и удаление текстовых заметок	обязательное
F4	Списки	создание списка и отдельных пунктов с возможностью отметки выполнения	обязательное
F5	Напоминания	создание напоминаний с датой и статусом	обязательное
F6	Папки	создание, переименование, удаление и фильтрация объектов по папке	обязательное
F7	Ассистент	ответ на обычные вопросы и использование сохраненного контекста	обязательное
F8	Команды	различение обычного вопроса и команды сохранения	обязательное
F9	Настройки	отображение профиля и статистики рабочего кабинета	желательное
F10	Поиск	поиск по названию и содержимому объектов	желательное
F11	Предпросмотр файлов	показ извлеченного текста, если формат поддерживается	желательное

2.4. Нефункциональные требования

Таблица 7 - Нефункциональные требования

Код	Требование	Содержание
N1	Простота интерфейса	основные разделы должны быть доступны из навигации, а основные действия - без лишних экранов
N2	Скорость ответа	операции CRUD должны выполняться без заметных задержек в локальной демонстрации
N3	Устойчивость к ошибкам API	ошибка ИИ-провайдера не должна приводить к потере пользовательского сообщения
N4	Переносимость	проект должен запускаться локально и в контейнерах
N5	Безопасность базового уровня	доступ к данным выполняется после авторизации, пароль хранится в виде хэша
N6	Расширяемость	архитектура должна позволять добавлять новые типы объектов и интеграции
N7	Кроссплатформенность	основные сценарии должны быть доступны через современный браузер на desktop и mobile

Нефункциональные требования в MVP не равны требованиям промышленной эксплуатации. Например, в проекте реализована авторизация и базовое разделение пользовательских данных, но не реализованы роли, аудит действий, резервное копирование и полноценная политика безопасности. Эти ограничения явно фиксируются, чтобы не завышать результат ВКР.

2.5. Приоритизация требований

Таблица 8 - Приоритизация требований MVP

Группа	Требования	Обоснование
Обязательные	чат, файлы, заметки, списки, напоминания, папки, авторизация, backend API, БД	без этих функций не демонстрируется основная идея личного секретаря
Желательные	поиск, настройки, предпросмотр файлов, статистика, адаптивные улучшения	повышают удобство, но не меняют ядро сценария
Возможные	семантический поиск, OCR, календарь, совместная работа, роли, мобильное приложение	важны для развития после защиты, но требуют отдельной архитектурной проработки

2.6. Критерии выбора инструментов

Технологический стек выбирался не абстрактно, а по фактическому коду проекта и требованиям MVP. Для frontend требовались компонентный подход, типизация, быстрый dev-сервер и возможность собрать SPA. Для backend требовались HTTP API, авторизация, работа с PostgreSQL, загрузка файлов и интеграция с внешним ИИ-провайдером.

Для базы данных важны связи между пользователем, папками, объектами и сообщениями. Поэтому хранение в JSON-файлах или только в памяти не подходит: такая модель усложнила бы фильтрацию, поиск, обновление и демонстрацию реального состояния приложения. PostgreSQL выбран как надежная реляционная БД с понятной моделью связей [11].

Отдельным критерием была воспроизводимость запуска. Наличие Docker Compose позволяет поднять web, api и db как согласованный набор сервисов. Это полезно для демонстрации, потому что снижает зависимость от ручной настройки локального окружения [13].

2.7. Сравнение технологических вариантов

Таблица 9 - Сравнение frontend-вариантов

Вариант	Плюсы	Минусы	Решение
React + Vite + TypeScript	быстрая сборка, компонентный подход, типизация, хорошо подходит для SPA	требуется отдельно организовать маршрутизацию и API-клиент	фактически выбран
Next.js	полный фреймворк, SSR и маршрутизация	избыточен для локального SPA-MVP, в проекте не используется	не выбран
Vue/Nuxt	понятная реактивность и экосистема	не соответствует фактическому коду проекта	не выбран
Angular	полный фреймворк и строгая структура	выше порог входа и больше шаблонного кода для MVP	не выбран
HTML/CSS/JS	минимум зависимостей	сложнее поддерживать крупный интерфейс и типизированный API	не выбран

Таблица 10 - Сравнение backend-вариантов

Вариант	Плюсы	Минусы	Решение
Node.js + Express + TypeScript	простая маршрутизация, хорошая интеграция с frontend-стеком, поддержка middleware	часть архитектурных решений нужно задавать самостоятельно	фактически выбран
NestJS	модульность, DI, строгая структура	для MVP может быть тяжелее и требует больше инфраструктуры	не выбран
Django/FastAPI	сильная серверная экосистема Python	увеличивает число языков и не соответствует текущему монорепозиторию	не выбран
Монолит без backend	быстрый старт	не подходит для БД, авторизации и ИИ-интеграции	не выбран

Таблица 11 - Сравнение вариантов хранения данных

Вариант	Плюсы	Минусы	Решение
PostgreSQL + Drizzle ORM	реляционная модель, миграции в TypeScript, типизированные запросы	требуется отдельный сервис БД	фактически выбран
SQLite	простая локальная БД	хуже подходит для контейнерной демонстрации и многопользовательской модели	не выбран
MySQL	распространенная реляционная БД	не используется в проекте и не дает преимущества для MVP	не выбран
JSON-файлы	легко начать	сложно обеспечить целостность, поиск и конкурентный доступ	не выбран

2.8. Выводы по главе

Во второй главе определены границы MVP, пользовательские истории, функциональные и нефункциональные требования. Основным принципом стало честное ограничение объема: в работу включаются функции, подтвержденные кодом проекта, а идеи развития выносятся в перспективы.

Фактический технологический стек проекта: React 19, Vite, TypeScript, Tailwind CSS, shadcn/Radix UI, TanStack Query и Wouter на клиенте; Node.js, Express 5, TypeScript, Zod и pino на сервере; PostgreSQL и Drizzle ORM для данных; Docker Compose для локального контейнерного запуска; OpenAI-compatible/OpenRouter/OpenAI API для ИИ-ответов.

Такой стек соответствует задаче ВКР: он позволяет собрать кроссплатформенное веб-приложение с типизированным API, хранением данных, адаптивным интерфейсом и демонстрационной интеграцией ассистента.

ГЛАВА 3. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ «ЛИЧНЫЙ СЕКРЕТАРЬ» / MINDVAULT

3.1. Общая архитектура системы

Архитектура приложения построена по клиент-серверной модели. Пользователь взаимодействует с React-приложением в браузере. Клиентская часть отправляет запросы к Express API, сервер проверяет авторизацию, выполняет операции с базой данных через Drizzle ORM, при необходимости обращается к ИИ-провайдеру и возвращает результат во frontend.

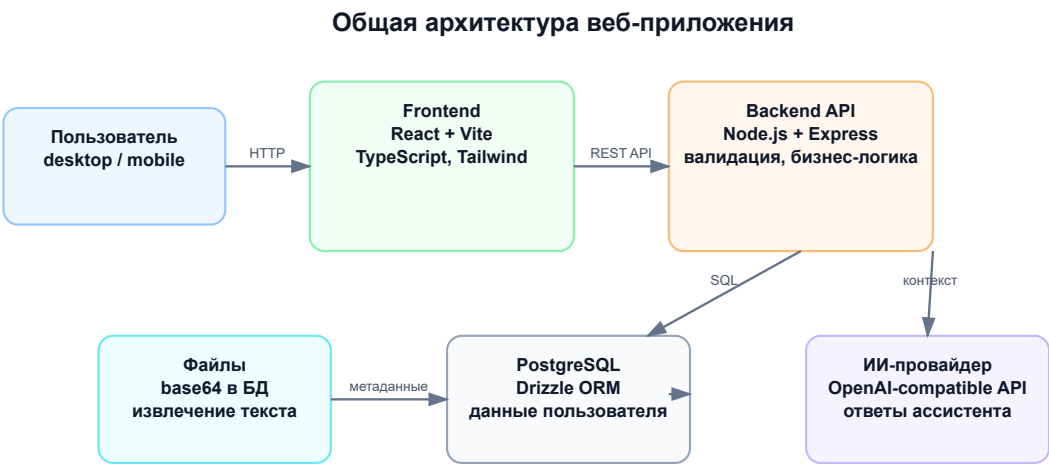


Рисунок 1 - Общая архитектура веб-приложения «Личный секретарь»

На рисунке 1 показано разделение ответственности. Frontend отвечает за состояние интерфейса, маршрутизацию и взаимодействие пользователя. Backend отвечает за бизнес-логику, безопасность доступа, хранение данных и интеграцию с ассистентом. PostgreSQL хранит пользователей, папки, объекты, диалоги и сообщения.

В архитектуре приложения важно, что ассистент не является единственным источником действий. Если пользователь создает заметку через интерфейс, операция выполняется обычным API-запросом и не зависит от ИИ-провайдера.

В архитектуре приложения обработка файлов вынесена на сервер, потому что именно backend должен извлечь текст, сохранить метаданные и подготовить данные для будущего контекстного поиска.

В архитектуре приложения ИИ используется как надстройка над сохраненными материалами. Такой подход позволяет сохранить базовую полезность приложения даже при недоступности внешнего провайдера.

3.2. Диаграмма развертывания

Для демонстрации проект запускается в контейнерах. Docker Compose описывает три основных сервиса: web, api и db. Web-сервис отдает клиентское приложение, api-сервис обслуживает REST API, db-сервис хранит PostgreSQL-данные. Такой вариант удобен для проверки ВКР, так как обеспечивает повторяемый запуск на другой машине при наличии Docker.

Диаграмма развертывания MindVault

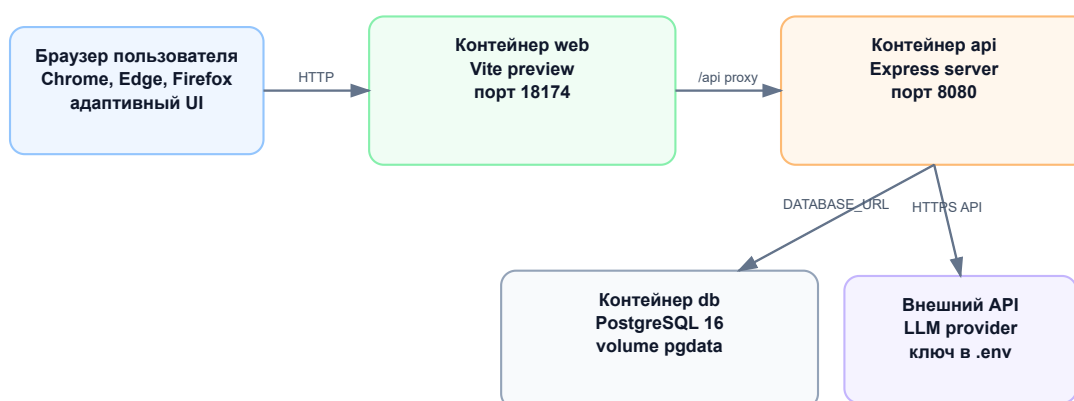


Рисунок 2 - Диаграмма развертывания MindVault

Контейнеризация не превращает MVP в промышленную систему, но помогает подготовить демонстрацию. В промышленном варианте потребуются HTTPS, доменное имя, резервное копирование БД, управление секретами и мониторинг. В рамках ВКР эти элементы описаны как перспективы, а не как уже реализованные функции.

3.3. Event modelling / timeline событий

Обработка сообщения проектировалась как последовательность событий. Такой подход помогает увидеть, в какой момент пользовательский ввод становится объектом базы данных, а в какой остается обычным сообщением чата. Timeline также показывает, где должно появляться подтверждение действия.



Рисунок 3 - Timeline обработки пользовательского сообщения

Если пользователь задает обычный вопрос, система сохраняет сообщение в истории и формирует ответ ассистента. Если ввод является явной командой, backend выполняет соответствующее действие: создает заметку, список или напоминание. Если ввод спорный, система не должна автоматически создавать объект, а должна предложить пользователю выбор.

3.4. Архитектура frontend

Frontend реализован как SPA на React и Vite [6], [7]. Основные страницы соответствуют разделам: чат, файлы, заметки, списки, напоминания, настройки и папки. Для маршрутизации используется Wouter, для серверного

состояния - TanStack Query, для визуальных элементов - Tailwind CSS, shadcn/ui и Radix UI.

Клиентская часть не содержит прямого доступа к базе данных. Все данные получаются через API-клиент, сгенерированный на основе OpenAPI-описания. Это снижает риск расхождения контрактов между frontend и backend, так как типы запросов и ответов формируются автоматически инструментом Orval [14], [15].

В интерфейсе используется единая логика карточек: заметка, файл, список и напоминание имеют разные поля, но визуально воспринимаются как элементы одного рабочего кабинета. Это решение важно для UX: пользователь видит не набор несвязанных страниц, а одно пространство с несколькими представлениями.

3.5. Архитектура backend

Backend реализован на Node.js и Express 5 [9], [10]. Сервер содержит маршруты авторизации, профиля, папок, объектов, поиска, статистики, диалогов и классификации сообщений. Валидация входных данных выполняется через Zod-схемы, сгенерированные по OpenAPI-спецификации.

Для авторизации используется JWT-токен, который frontend хранит на стороне браузера. Пароль пользователя не хранится в открытом виде: при регистрации создается хэш. Все операции с папками и объектами проверяют принадлежность данных текущему пользователю.

Серверная часть отвечает за важное правило: ассистент не должен подтверждать создание, изменение или удаление объекта, если backend не выполнил операцию успешно. Это правило зафиксировано в системной инструкции ассистента и поддержано логикой обработки действий.

3.6. Модель данных



Рисунок 4 - Схема базы данных

Таблица 12 - Сущности базы данных

Таблица	Назначение	Ключевые поля
users	учетные записи пользователей	id, email, passwordHash, fullName, avatarUrl, createdAt, updatedAt
folders	пользовательские и системные папки	id, userId, name, color, icon, isSystem
items	единая таблица заметок, файлов, списков и напоминаний	id, userId, folderId, type, title, content, fileData, reminderAt, status
conversations	диалоги ассистента	id, userId, title, createdAt, updatedAt
messages	сообщения в диалогах	id, conversationId, role, content, metadata, createdAt

Использование единой таблицы items для разных типов объектов упрощает общие операции: поиск, фильтрацию по папке, вывод недавних материалов и подсчет статистики. Различия между типами выражаются через поле type и дополнительные поля. Для файла заполняются originalFilename, mimeType, fileSize и fileData; для напоминания - reminderAt; для списка content хранит структурированные пункты.

У модели есть осознанное ограничение MVP: файл сохраняется в базе данных в виде base64. Это упрощает демонстрацию и не требует отдельного

объектного хранилища, но для промышленного варианта лучше вынести большие файлы в S3-совместимое хранилище или файловый сервис, а в БД оставлять метаданные и ссылку.

3.7. Проектирование обработки сообщений



Рисунок 5 - Алгоритм различения обычного запроса и команды на сохранение

Алгоритм обработки сообщений строится вокруг намерения пользователя. Если сообщение выглядит как вопрос, оно обрабатывается как обычный чат. Если есть явная команда «сохрани заметку», «создай список» или «напомни», backend может создать объект. Если сообщение неоднозначно, например «заметка список напоминание», безопаснее не выполнять действие автоматически.

Причина такого решения практическая. Пользователь может проверять возможности системы, задавать вопрос о типах объектов или случайно написать несколько ключевых слов. Автоматическое сохранение в таких случаях ухудшает качество данных и создает недоверие к приложению. Поэтому спорные случаи переводятся в режим уточнения.

Подтверждение действия должно появляться только после успешной операции на backend. Если объект не создан из-за ошибки валидации, отсутствия папки, сбоя базы данных или ошибки авторизации, ассистент не имеет права писать «готово». Это правило снижает риск потери важных напоминаний.

3.8. Проектирование работы с файлами

Файловая подсистема проектировалась как часть общего рабочего кабинета. Пользователь загружает файл, сервер сохраняет метаданные и содержимое, пытается извлечь текст и добавляет файл в папку. Если текст извлечен, он может использоваться ассистентом как контекст при ответе.

В MVP поддерживается извлечение текста из некоторых текстовых документов. Для DOCX используется библиотека Mammoth, для PDF - pdf-parse [25], [26]. Неподдерживаемые или плохо читаемые файлы сохраняются с метаданными, но ассистент не должен выдумывать их содержимое. В интерфейсе такие случаи должны быть обозначены как ограничение чтения.

Работа с файлами также влияет на безопасность. Пользовательские файлы доступны только владельцу через авторизованные запросы. В текущей версии не реализованы антивирусная проверка, ограничение типов по белому списку и внешнее объектное хранилище, поэтому эти задачи относятся к дальнейшему развитию.

3.9. Проектирование интерфейса



Рисунок 6 - Структура пользовательского интерфейса

Интерфейс проектировался как рабочий кабинет с боковой навигацией на desktop и компактным меню на мобильном экране. В основной области отображается выбранный раздел, а внизу чата находится панель ввода. Такой вариант сохраняет привычность мессенджера, но добавляет структуру информационной системы.

Визуальный стиль выбран спокойным: светлый фон, карточки, акцентный синий цвет для активного раздела и действий. Задача интерфейса - не рекламировать продукт, а помогать пользователю быстро понять, где находятся чат, заметки, списки, файлы и напоминания. При проектировании учитывались базовые эвристики удобства и доступности [21], [27].

3.10. Выводы по главе

В третьей главе спроектированы архитектура, развертывание, timeline обработки сообщений, frontend, backend, модель данных, работа с файлами и интерфейс. Проектирование показало, что ключевой особенностью приложения является не отдельная функция, а согласованность между чатовым вводом, структурированными объектами и ассистентом.

Принятые решения соответствуют границам MVP. Единая таблица items упрощает работу с разными типами материалов, Docker Compose упрощает демонстрационный запуск, а алгоритм различения намерений снижает риск ошибочного сохранения пользовательских сообщений.

ГЛАВА 4. РЕАЛИЗАЦИЯ СИСТЕМЫ

4.1. Структура проекта

Исходный код MindVault организован как рnrn-монорепозиторий. В нем выделены приложения frontend и backend, а также общие библиотеки для базы данных, API-спецификации, сгенерированного клиента и интеграции с ИИ. Такая структура уменьшает дублирование типов и позволяет согласованно менять контракт между сервером и клиентом.

Таблица 13 - Структура проекта

Путь	Назначение
artifacts/mindvault	клиентское приложение React/Vite
artifacts/api-server	серверное приложение Express
lib/db	схема PostgreSQL и Drizzle ORM
lib/api-spec	OpenAPI-описание REST API
lib/api-client-react	сгенерированный клиент для frontend
lib/api-zod	сгенерированные Zod-схемы для backend
lib/integrations-gemini-ai	адаптер к OpenAI-compatible/OpenRouter/OpenAI API
docker-compose.yml	локальное контейнерное развертывание web, api и db

Название библиотеки интеграции с ИИ исторически содержит слово gemini, однако фактический код работает с OpenAI-compatible API и поддерживает ключи OPENROUTER_API_KEY или OPENAI_API_KEY. В тексте работы поэтому используется формулировка «OpenAI-compatible/OpenRouter/OpenAI API», чтобы не исказить реальный стек проекта.

4.2. Реализация клиентской части

Клиентская часть реализует защищенные маршруты. Если токен отсутствует, пользователь перенаправляется на страницу авторизации. После входа открывается основной интерфейс с навигацией, разделами и профилем. На рисунке 7 показан главный экран приложения MindVault с ответом ассистента и источниками из сохраненного контекста.

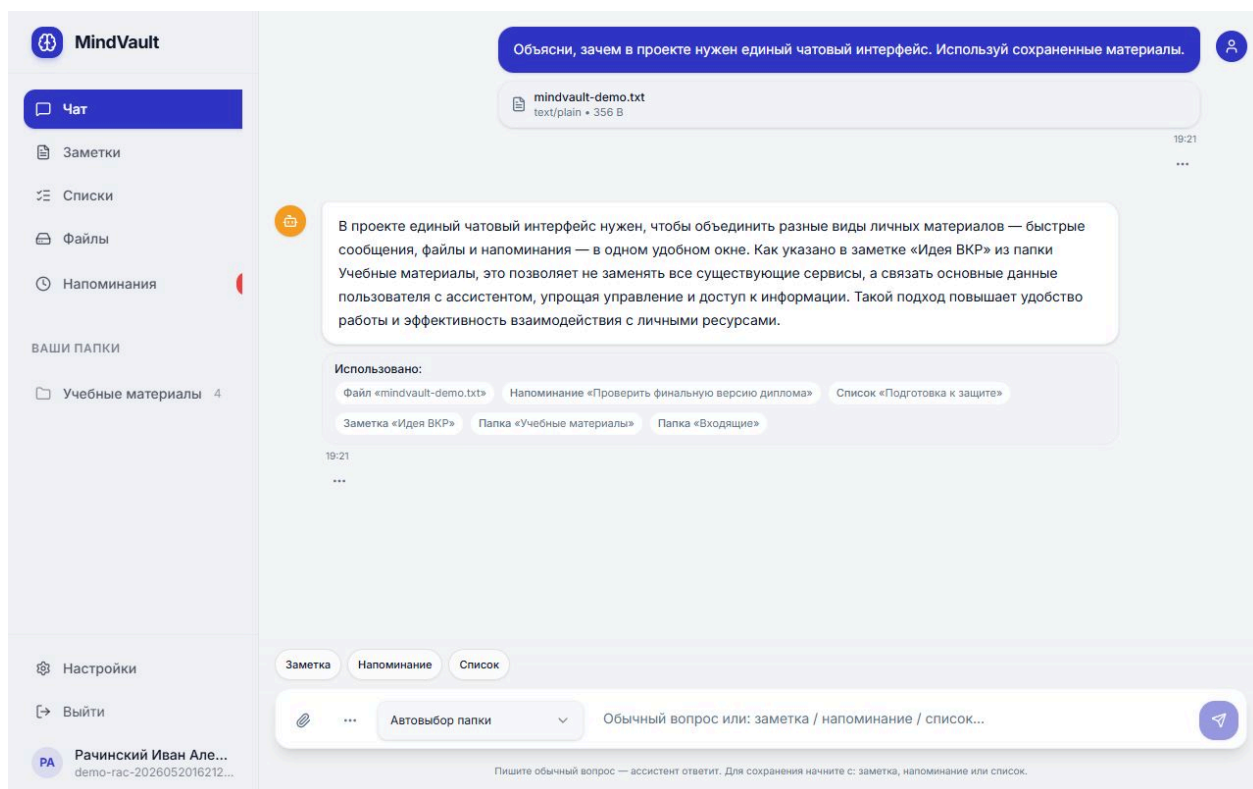


Рисунок 7 - Главный экран приложения MindVault

Работа с серверным состоянием организована через TanStack Query. Это позволяет отделить локальное состояние компонентов от данных, полученных с API. При создании или обновлении объекта соответствующие запросы инвалидируются, и интерфейс получает актуальные данные.

UI-компоненты собраны в едином стиле. Для кнопок, карточек, полей ввода, диалогов и выпадающих списков используются компоненты shadcn/ui и Radix UI, стилизованные через Tailwind CSS. Такой подход ускорил разработку, но не снял необходимость вручную приводить разделы к единой логике.

4.3. Реализация серверной части

Серверная часть содержит Express-приложение с маршрутом /api. Основные группы маршрутов: auth, folders, items, gemini и health. Входные данные валидируются Zod-схемами, а ошибки возвращаются в виде JSON. Для логирования используется Pino.

Таблица 14 - Основные API-маршруты

Группа	Маршруты	Назначение
auth	POST /auth/register, POST /auth/login, GET /auth/me	регистрация, вход и получение профиля
folders	GET/POST/PATCH/DELETE /folders	CRUD-папок пользователя
items	GET/POST/PATCH/DELETE /items, POST /items/upload	работа с заметками, файлами, списками и напоминаниями
search	GET /search	поиск по названию и содержимому
stats	GET /stats/workspace, /stats/recent, /stats/reminders/upcoming	статистика и недавние объекты
gemini	GET/POST /gemini/conversations, /messages, /classify	диалоги, сообщения и классификация намерений
health	GET /healthz	проверка доступности сервера

Серверная логика проверяет принадлежность папок и объектов текущему пользователю. Например, если пользователь передает `folderId` при создании заметки или файла, backend сначала убеждается, что папка существует и принадлежит этому пользователю. Это базовое, но важное правило безопасности.

4.4. Реализация базы данных

Drizzle ORM используется для описания схемы и выполнения типизированных запросов. Таблицы `users`, `folders`, `items`, `conversations` и `messages` отражают модель, описанную в главе 3. В коде запросы строятся через функции `eq`, `and`, `desc`, `asc`, `ilike` и `sql`, что сохраняет связь с SQL и одновременно дает типизацию TypeScript [12].

Для MVP выбран прагматичный вариант: разные пользовательские объекты хранятся в одной таблице `items`. Это упрощает API и интерфейс: один маршрут может возвращать все объекты, фильтровать их по `type`, `folderId` и `status`, а frontend может строить разные представления на основе одного контракта.

При реализации важно было не смешивать системные и пользовательские папки. Системные папки создаются при регистрации, а пользовательские можно создавать, переименовывать и удалять. При удалении

пользовательской папки связанные объекты не удаляются, а переводятся в состояние без папки, что безопаснее для данных.

4.5. Реализация файлового хранилища

Загрузка файла выполняется через POST /items/upload. Клиент передает имя файла, MIME-тип, размер, base64-содержимое и при необходимости папку. Сервер извлекает текст, сохраняет объект типа file и возвращает карточку файла. Раздел файлов показан на рисунке 8.

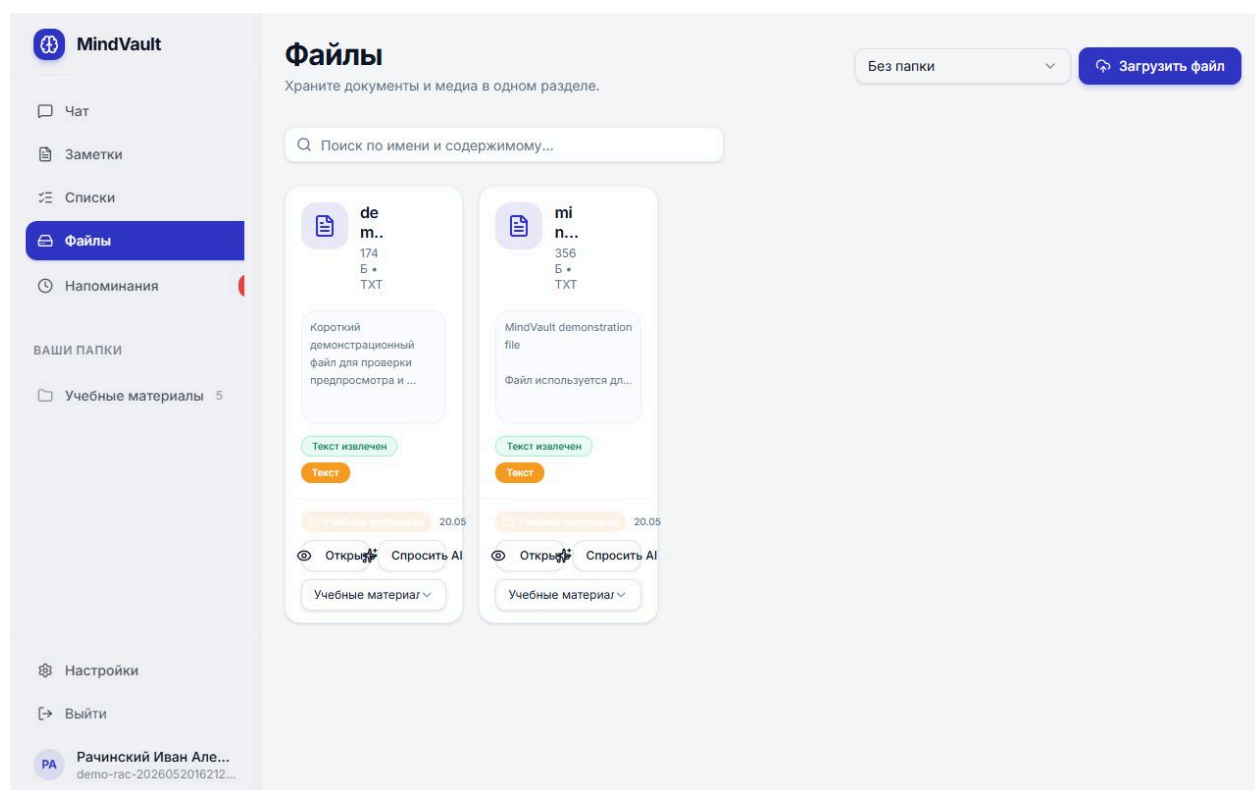


Рисунок 8 - Раздел файлов

В текущей реализации текстовые файлы и поддерживаемые документы могут получить текстовый предпросмотр. На рисунке 14 показана работа с загруженным файлом: карточка содержит метаданные, фрагмент текста, признак извлечения и действие «Спросить AI». Для длинных имен файлов выявлено ограничение верстки: в узкой карточке имя может переноситься слишком агрессивно. Это не нарушает хранение данных, но требует доработки интерфейса.

4.6. Реализация папок

Папки используются как общий механизм организации материалов. Пользователь может создать папку, переименовать ее, удалить и отфильтровать содержимое. В боковой панели отображается список пользовательских папок и количество активных объектов внутри. Это помогает не перегружать основной экран и быстро перейти к нужному контексту.

При создании нового пользователя backend автоматически создает системные папки. В ходе проверки проекта было выявлено, что часть строк в отдельных участках исходного кода ранее была записана с поврежденной кодировкой. Для демонстрационного пользователя была создана пользовательская папка «Учебные материалы» с корректным названием, а само ограничение отражено в тестировании как требующее аккуратной локализационной доработки.

4.7. Реализация заметок, списков и напоминаний

Раздел заметок предназначен для хранения текстовых материалов. Пользователь может создать запись вручную или через чатовую команду. Карточка заметки содержит заголовок, фрагмент содержимого, дату и папку. Реальный раздел заметок показан на рисунке 9.

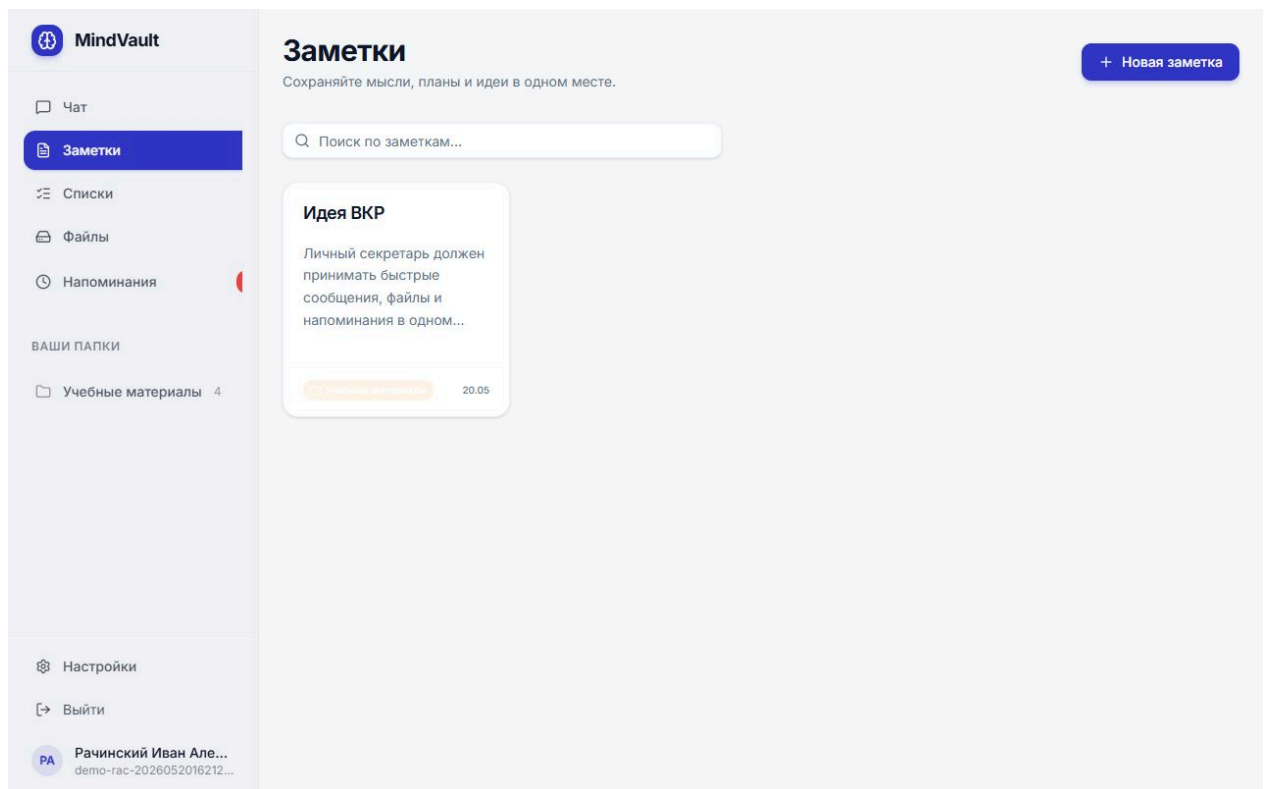


Рисунок 9 - Раздел заметок

Списки реализованы как объект типа `list`. Содержимое списка хранится в JSON-структуре с пунктами и признаком выполнения. Такой вариант не требует отдельной таблицы `list_items` в MVP, но позволяет отображать чек-лист и менять состояние пунктов. Раздел списков показан на рисунке 10.

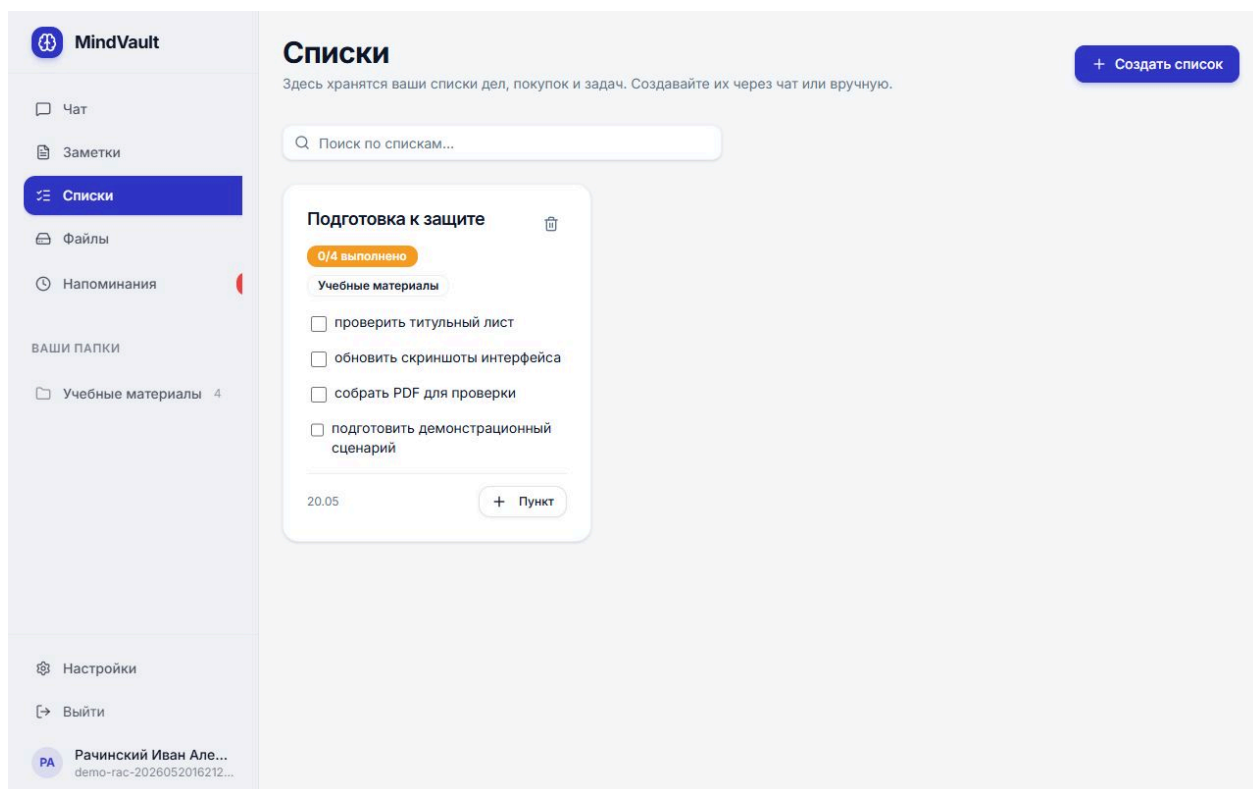


Рисунок 10 - Раздел списков

Напоминания реализованы как объект типа `reminder` с полем `reminderAt` и статусом. В интерфейсе показывается дата, содержимое и папка. Раздел напоминаний показан на рисунке 11.

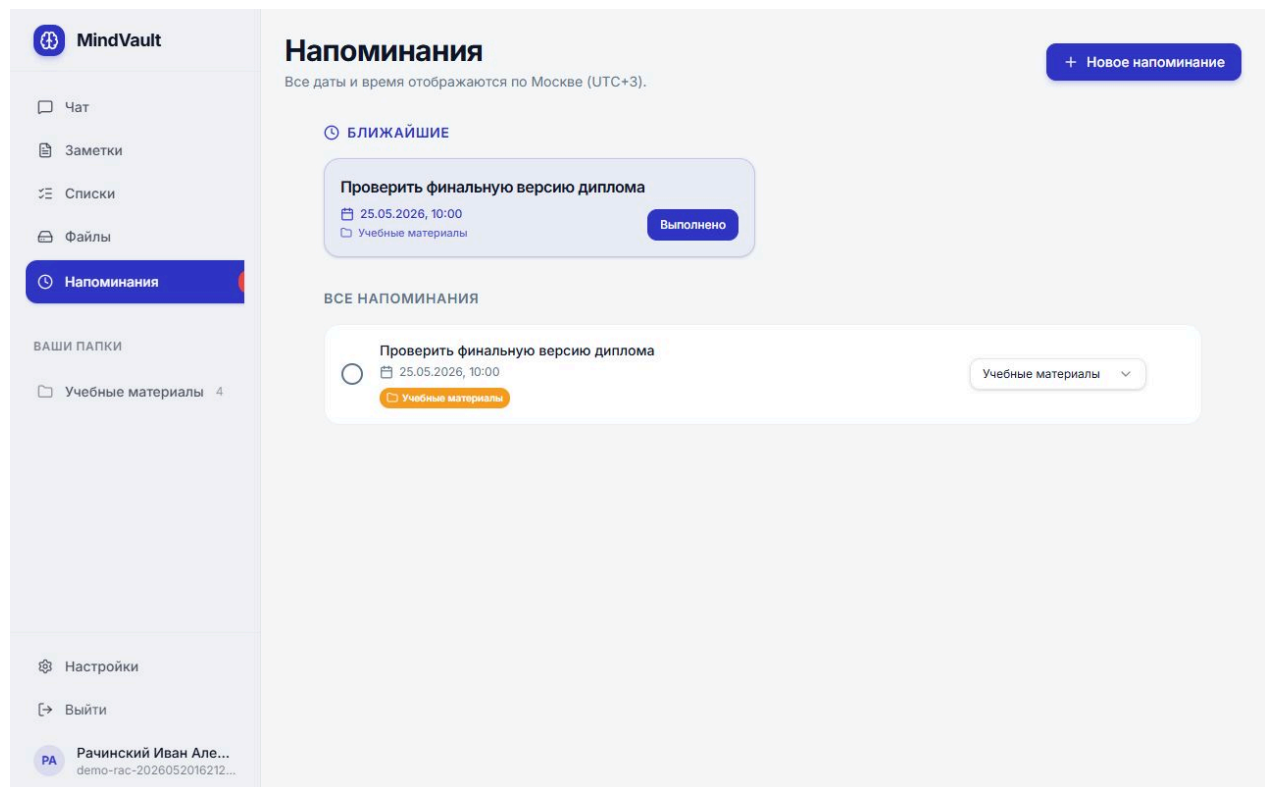


Рисунок 11 - Раздел напоминаний

4.8. Реализация логики ассистента

Ассистент работает с диалогами и сообщениями. Пользовательское сообщение сохраняется, затем сервер формирует контекст из сохраненных объектов и передает его во внешний ИИ-провайдер. Если провайдер недоступен, backend возвращает офлайн-ответ и не теряет сообщение. Такой подход важен для устойчивости демонстрационной версии.

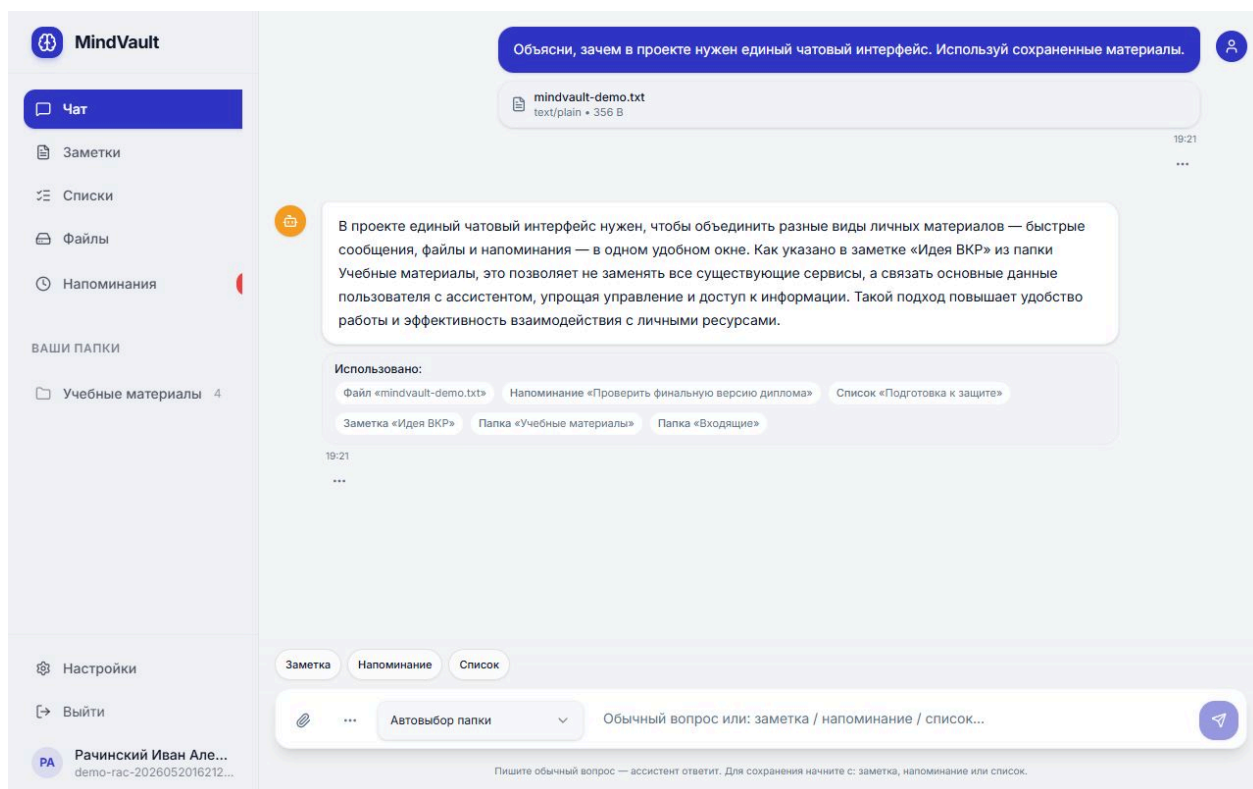


Рисунок 12 - Пример работы ассистента

Отдельный маршрут `/gemini/classify` отвечает за классификацию пользовательского ввода и выполнение действий. В коде предусмотрены режимы `reply_only`, `saved`, `suggest_actions` и `action_executed`. Это позволяет не сводить работу ассистента к простому текстовому ответу, а связывать ее с реальными операциями приложения.

При разработке возникла проблема: ассистент мог сохранять слишком много сообщений как заметки или напоминания. Для MVP был выбран более осторожный вариант. Явные команды выполняются, обычные вопросы остаются чатом, а спорные вводы переводятся в уточнение. Такой подход снижает количество ошибочных записей.

4.9. Реализация адаптивного интерфейса

Кроссплатформенность приложения обеспечивается веб-интерфейсом. На desktop используется боковая навигация, а на мобильном экране интерфейс сжимается до компактного режима. На рисунке 12 показан мобильный вид чата: сохраняются сообщения, источники, кнопки команд и поле ввода.

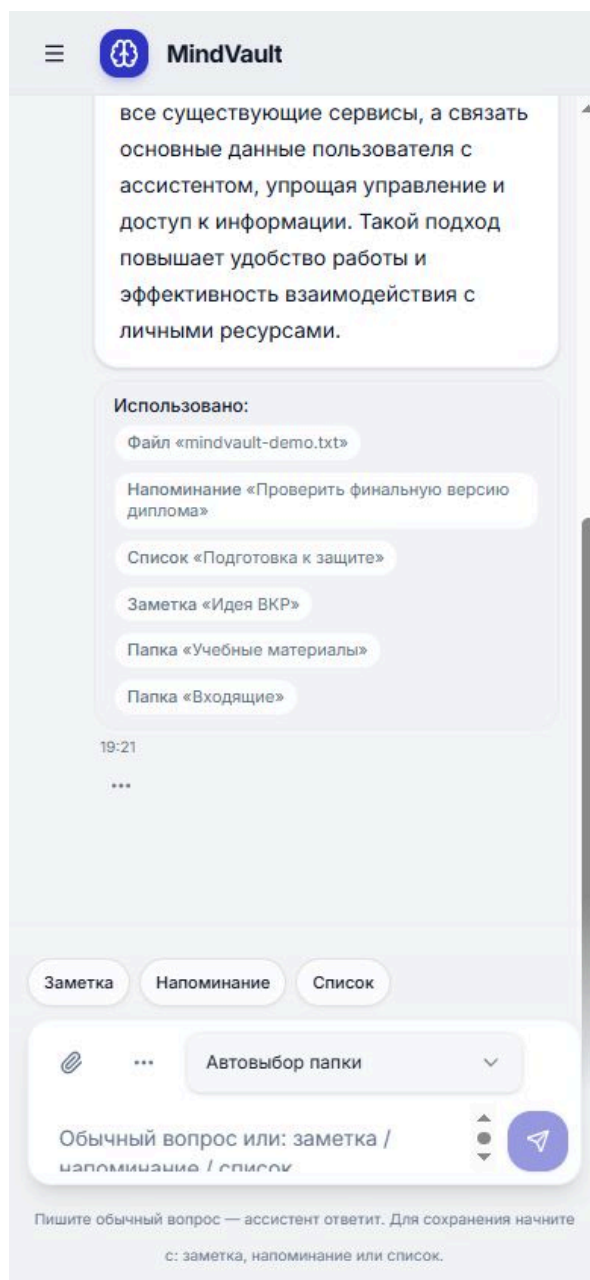


Рисунок 13 - Мобильная версия интерфейса

При тестировании мобильного вида отдельно проверялось, что поле ввода не перекрывает кнопку отправки и что пользователь может продолжать основной сценарий. Полноценное нативное мобильное приложение не входит в MVP, но адаптивный веб-интерфейс позволяет использовать систему на смартфоне.

4.10. Развертывание и запуск

Проект запускается через Docker Compose. В демонстрационном окружении web-сервис доступен на порту 18174, api-сервис - на порту 8080,

база данных PostgreSQL работает в отдельном контейнере. Проверка `/api/healthz` возвращает статус `ok`, что подтверждает доступность backend.

Для разработки также используются `pnpm workspace` и TypeScript-конфигурации. Проверка `typescheck` проходит по библиотекам, `backend`, `frontend` и вспомогательным пакетам. Серверные тесты проверяют командный парсер, включая спорные вводы и список покупок.

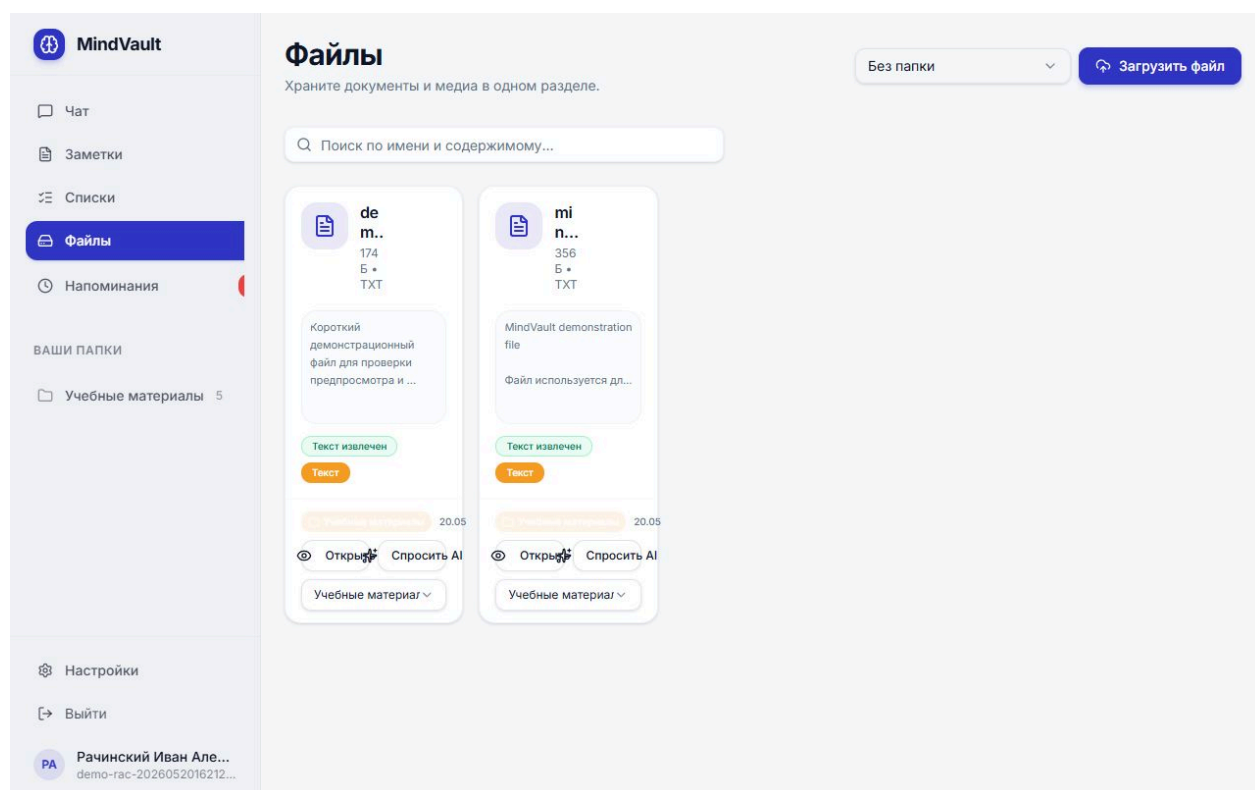


Рисунок 14 - Работа с загруженным файлом

4.11. Сложности при разработке и принятые решения

Первая сложность была связана с поведением ассистента. Если слишком доверять языковой модели, она может подтверждать действие раньше, чем оно выполнено. Решением стало правило: сообщение «создано» или «готово» допустимо только после успешной backend-операции.

Вторая сложность - различие обычной просьбы и команды сохранения. Пользователь может написать «напомни, что такое MVP?» как вопрос, а может написать «напомни завтра проверить MVP» как задачу. В MVP была реализована осторожная логика, которая снижает риск автосохранения спорных сообщений.

Третья сложность - единый стиль разделов. Файлы, заметки, списки и напоминания имеют разные данные, но пользователь должен воспринимать их как элементы одного кабинета. Поэтому карточки разделов были приведены к сходной структуре.

Четвертая сложность - файлы. Не каждый файл можно корректно прочитать как текст. В таких случаях приложение сохраняет файл и метаданные, но не должно выдумывать содержимое. Для MVP это честное ограничение, а не ошибка концепции.

Пятая сложность - подготовка демонстрации. Контейнерный запуск помогает показать проект на сервере или локальной машине, но для промышленной демонстрации потребуются HTTPS, домен, резервное копирование и управление секретами.

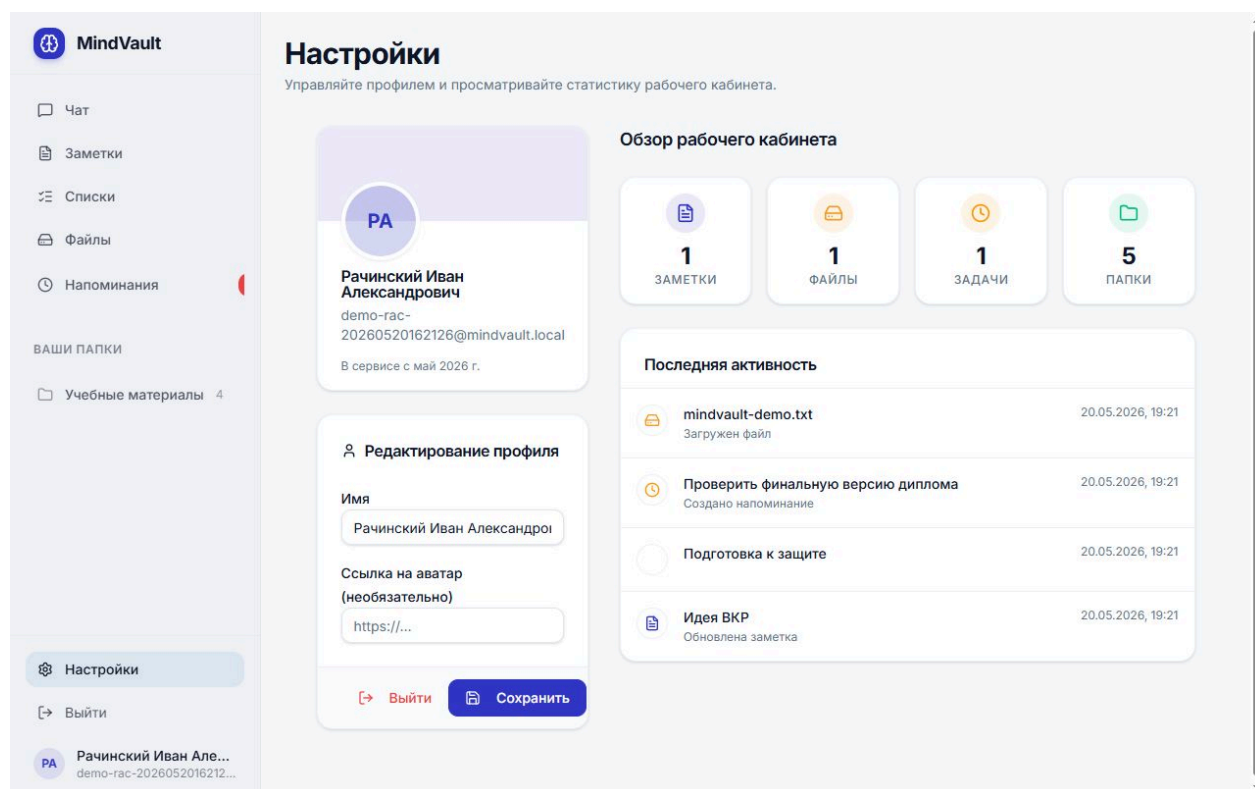


Рисунок 15 - Раздел настроек

4.12. Выводы по главе

В четвертой главе описана реализация основных подсистем приложения MindVault. Фактический результат соответствует теме ВКР: разработано кроссплатформенное веб-приложение «Личный секретарь» с использованием

ИИ, содержащее frontend, backend, базу данных, загрузку файлов, структурированные материалы и ассистента.

Реализация имеет ограничения MVP, но они не скрываются. Часть возможностей, например OCR, векторный поиск, внешнее файловое хранилище и промышленная безопасность, относится к дальнейшему развитию. Такой подход позволяет честно показать достигнутый результат и не приписывать проекту функции, которых в коде нет.

ГЛАВА 5. ТЕСТИРОВАНИЕ И АНАЛИЗ РЕЗУЛЬТАТОВ

5.1. Методика тестирования

Тестирование выполнялось как сочетание проверки автоматических команд, ручного функционального тестирования и визуальной проверки интерфейса. Для backend использовались имеющиеся тесты командного парсера и typescheck проекта. Для пользовательских сценариев приложение было запущено локально в контейнерах, после чего через API был создан демонстрационный пользователь и набор данных.

Статусы тест-кейсов разделены на три группы: «пройдено», «частично пройдено» и «требуется доработки». Такой подход выбран намеренно: если функция работает с ограничениями MVP, безопаснее указать частичный статус, чем завышать результат.

5.2. Функциональное тестирование

Таблица 15 - Проверка базовых функций

№	Сценарий	Ожидаемый результат	Статус
1	Создание заметки через раздел заметок	заметка появляется в разделе и сохраняется в БД	пройдено
2	Создание заметки через чат	явная команда создает объект или предлагает действие	частично пройдено
3	Создание списка	создается список с пунктами	пройдено
4	Создание напоминания	создается объект reminder с датой	пройдено
5	Загрузка файла	файл сохраняется с метаданными и содержимым	пройдено
6	Отображение файла в разделе файлов	видна карточка файла и текстовый предпросмотр	пройдено
7	Перемещение материала в папку	folderId объекта изменяется	пройдено
8	Переименование папки	новое имя отображается в навигации	пройдено
9	Удаление папки	папка удаляется, объекты не теряются	пройдено

5.3. Тестирование интерфейса

Интерфейс проверялся по основным разделам. В результате подтверждено наличие разделов чата, файлов, заметок, списков, напоминаний

и настроек. Разделы используют единый визуальный стиль, но для карточек файлов обнаружено ограничение переноса длинных имен.

Таблица 16 - Проверка интерфейса

Сценарий	Результат	Статус
Раздел списков имеет кнопку создания и единый стиль	кнопка «Создать список» присутствует, карточка списка отображает пункты	пройдено
Название вкладки браузера MindVault	заголовок приложения соответствует рабочему названию	пройдено
Иконка сайта во вкладке браузера	favicon присутствует в проекте	пройдено
Длинное имя файла в карточке	имя сохраняется, но перенос может быть неудачным	требуется доработки
Единый стиль карточек	карточки разделов визуально согласованы	пройдено

5.4. Тестирование адаптивности

Таблица 17 - Проверка адаптивности

Сценарий	Ожидаемый результат	Статус
Открытие на desktop	видна боковая навигация и рабочая область	пройдено
Открытие на мобильной ширине	интерфейс переходит в компактный режим	пройдено
Мобильное меню	доступ к основным разделам сохраняется	пройдено
Поле ввода на мобильном экране	поле и кнопка отправки не перекрывают друг друга	пройдено
Карточки на малой ширине	карточки остаются доступными, но отдельные длинные строки требуют улучшения	частично пройдено

5.5. Тестирование ассистента

Таблица 18 - Проверка ассистента

№	Сценарий	Результат	Статус
1	Обычный вопрос ассистенту без сохранения	ответ появляется в чате, объект note/reminder не создается	пройдено
2	Вопрос с сохраненным файлом	ассистент использует файл и сохраненные материалы как контекст	пройдено
3	Ошибка API ИИ	сообщение не теряется, возвращается офлайн-ответ	пройдено
4	Подтверждение действия	ответ о сохранении должен зависеть от результата backend	пройдено
5	Работа с несколькими источниками	в ответе отображаются использованные объекты	пройдено

5.6. Тестирование нестандартных вводов

Таблица 19 - Проверка нестандартных вводов

Ввод	Ожидаемая реакция	Статус
«заметка список напоминание»	не выполнять автоматическое сохранение без уточнения	пройдено тестами парсера
Пустое сообщение	кнопка отправки недоступна или сервер не создает объект	пройдено
Очень длинный текст	сообщение обрабатывается, но может быть ограничено UI и лимитами API	частично пройдено
Неподдерживаемый файл	файл сохраняется, текст не выдумывается	частично пройдено
Команда переместить последний объект	при наличии папки объект переносится	пройдено

5.7. Ограничения текущей версии

Таблица 20 - Ограничения MVP

Ограничение	Проявление	Влияние
Нет семантического поиска	поиск основан на обычном тексте и контексте	не мешает демонстрации MVP
Файлы хранятся base64 в PostgreSQL	большие файлы увеличивают БД	для промышленной версии нужно объектное хранилище
Нет OCR	изображения и сканы не читаются как текст	нужно добавить отдельную подсистему
Нет календарной интеграции	напоминания остаются внутри приложения	требуется дальнейшая интеграция
Нет совместной работы	данные принадлежат одному пользователю	подходит для личного MVP
Базовая безопасность	есть авторизация, но нет ролей и аудита	достаточно для учебной демонстрации
Локализация требует проверки	часть служебных строк в коде имеет следы поврежденной кодировки	нужно исправить перед публичной эксплуатацией

5.8. Сравнение результата с требованиями

Таблица 21 - Сравнение результата с требованиями

Требование	Результат	Статус
Чат	реализован основной чат с историей и ответом ассистента	выполнено
Файлы	реализована загрузка, хранение и предпросмотр текста	выполнено с ограничениями
Заметки	реализован раздел заметок и CRUD-логика	выполнено
Списки	реализован раздел списков и пункты	выполнено
Напоминания	реализован раздел напоминаний и дата	выполнено
Папки	реализованы пользовательские папки и фильтрация	выполнено
Ассистент	реализован ответ с контекстом и fallback при ошибке	выполнено
Адаптивность	desktop и mobile сценарии проверены	выполнено
Контейнерный запуск	web, api и db запускаются через Docker Compose	выполнено

Сравнение показывает, что цель MVP достигнута. Не все желательные функции доведены до промышленного уровня, однако основные сценарии приложения «Личный секретарь» реализованы и проверены. Особенно

важным результатом является связка файлов, заметок, списков, напоминаний и ассистента в одном интерфейсе.

5.9. Выводы по главе

В пятой главе проведено тестирование основных функций, интерфейса, адаптивности, ассистента и нестандартных вводов. Большинство базовых сценариев прошло проверку. Ограничения MVP зафиксированы явно и не противоречат демонстрации основной идеи.

Наиболее важные направления доработки: улучшение обработки длинных имен файлов, полная проверка локализации, добавление семантического поиска, перенос файлового содержимого во внешнее хранилище и усиление безопасности. Эти задачи логично выполнять после защиты как развитие проекта.

ЗАКЛЮЧЕНИЕ

В ходе выпускной квалификационной работы была достигнута цель: разработано кроссплатформенное веб-приложение «Личный секретарь» с использованием ИИ. Приложение получило рабочее название MindVault. Официальная тема работы сохранена в формулировке «Разработка кроссплатформенного веб-приложения «Личный секретарь» с использованием ИИ», а название MindVault используется как название разработанного продукта.

В работе проведен анализ предметной области и проблемы разрозненного хранения личных данных. Рассмотрены аналоги: Google Drive, Яндекс Диск, Dropbox, Notion, Obsidian, Evernote, Telegram Saved Messages, Todoist и ChatGPT. Анализ показал, что существующие решения хорошо закрывают отдельные задачи, но не дают единого MVP-сценария личного секретаря с чатовым входом, структурированными объектами и ассистентом.

На основе анализа были сформулированы требования к системе, user story, функциональные и нефункциональные требования. Были определены границы MVP: чат, файлы, заметки, списки, напоминания, папки, настройки, backend API, база данных, ассистент, адаптивный интерфейс и контейнерный запуск.

Проверка исходного кода позволила установить фактический стек проекта. Клиентская часть реализована на React, Vite и TypeScript с использованием Tailwind CSS, shadcn/Radix UI, TanStack Query и Wouter. Серверная часть реализована на Node.js, Express и TypeScript. Для хранения данных используется PostgreSQL и Drizzle ORM. Для запуска используется Docker Compose. Интеграция с ИИ выполнена через OpenAI-compatible/OpenRouter/OpenAI API.

В проектной части разработаны архитектурные схемы, диаграмма развертывания, модель данных, timeline обработки пользовательского сообщения, алгоритм различения обычного запроса и команды сохранения, а также структура пользовательского интерфейса. Особое внимание уделено правилу: ассистент не должен подтверждать действие, если backend не выполнил операцию успешно.

В реализации описаны клиентская и серверная части, работа с базой данных, файловым хранилищем, папками, заметками, списками, напоминаниями и ассистентом. Для демонстрации подготовлены реальные скриншоты запущенного приложения, включая главный чат, файлы, заметки, списки, напоминания, мобильный вид, пример ответа ассистента, работу с загруженным файлом и настройки.

Тестирование показало, что основные сценарии работают: пользователь может войти в приложение, создать папку, сохранить заметку, список, напоминание, загрузить файл, увидеть файл в разделе, задать обычный вопрос ассистенту и получить ответ с учетом сохраненного контекста. Также проверены нестандартные вводы и адаптивность.

Ограничения текущей версии описаны честно. В MVP отсутствуют семантический поиск, векторная база данных, OCR, календарные интеграции, совместная работа, роли пользователей, промышленный HTTPS-контур и резервное копирование. Кроме того, перед публичной эксплуатацией нужно завершить локализационную проверку и улучшить поведение длинных имен файлов в карточках.

Перспективы развития включают внедрение семантического поиска и векторной базы данных, улучшенное чтение PDF, DOCX и изображений, OCR, календарную интеграцию, совместную работу, расширенную безопасность, роли пользователей, мобильное приложение, HTTPS и домен для промышленной демонстрации, а также резервное копирование данных.

Таким образом, результат работы соответствует поставленной цели и демонстрирует инженерно завершенный MVP. Проект не пытается заменить все существующие сервисы, а решает конкретную задачу: дать пользователю единый интерфейс, куда можно быстро отправить информацию, а затем найти, структурировать и использовать ее с помощью ассистента.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Костромской государственный университет. Правила оформления текстовых документов: методические рекомендации. Кострома, 2017.
2. Костромской государственный университет. Методические указания по выполнению выпускной квалификационной работы для направлений подготовки в области информационных систем.
3. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления.
4. ГОСТ Р 7.0.5-2008. Библиографическая ссылка. Общие требования и правила составления.
5. ГОСТ 7.1-2003. Библиографическая запись. Библиографическое описание. Общие требования и правила составления.
6. React Documentation. URL: <https://react.dev/> (дата обращения: 20.05.2026).
7. Vite Documentation. URL: <https://vite.dev/guide/> (дата обращения: 20.05.2026).
8. TypeScript Handbook. URL: <https://www.typescriptlang.org/docs/handbook/intro.html> (дата обращения: 20.05.2026).
9. Node.js API Documentation. URL: <https://nodejs.org/api/> (дата обращения: 20.05.2026).
10. Express 5.x API Reference. URL: <https://expressjs.com/en/api.html> (дата обращения: 20.05.2026).
11. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/> (дата обращения: 20.05.2026).
12. Drizzle ORM Documentation. URL: <https://orm.drizzle.team/docs/overview> (дата обращения: 20.05.2026).
13. Docker Compose Documentation. URL: <https://docs.docker.com/compose/> (дата обращения: 20.05.2026).

14. OpenAPI Specification. URL: <https://spec.openapis.org/oas/latest.html> (дата обращения: 20.05.2026).
15. Orval Documentation. URL: <https://orval.dev/docs> (дата обращения: 20.05.2026).
16. TanStack Query Documentation. URL: <https://tanstack.com/query/docs> (дата обращения: 20.05.2026).
17. Tailwind CSS Documentation. URL: <https://tailwindcss.com/docs> (дата обращения: 20.05.2026).
18. shadcn/ui Documentation. URL: <https://ui.shadcn.com/docs> (дата обращения: 20.05.2026).
19. Zod Documentation. URL: <https://zod.dev/> (дата обращения: 20.05.2026).
20. OWASP Top 10:2021. URL: <https://owasp.org/Top10/2021/> (дата обращения: 20.05.2026).
21. Web Content Accessibility Guidelines (WCAG) 2.2. URL: <https://www.w3.org/TR/WCAG22/> (дата обращения: 20.05.2026).
22. MDN Web Docs. Fetch API. URL: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API (дата обращения: 20.05.2026).
23. OpenAI API Reference. Responses API. URL: <https://platform.openai.com/docs/api-reference/responses> (дата обращения: 20.05.2026).
24. OpenRouter API Reference. URL: <https://openrouter.ai/docs/api/reference/overview/> (дата обращения: 20.05.2026).
25. Mammoth.js Documentation. URL: <https://github.com/mwilliamson/mammoth.js> (дата обращения: 20.05.2026).
26. pdf-parse package documentation. URL: <https://www.npmjs.com/package/pdf-parse> (дата обращения: 20.05.2026).
27. Nielsen Norman Group. Usability Heuristics for User Interface Design. URL: <https://www.nngroup.com/articles/ten-usability-heuristics/> (дата обращения: 20.05.2026).